RescueMesh

A Cloud-P2P Environment for Smart-Campus Emergency Response and Resource Coordination

CSCE 4411 – Term Project
Fall 2026

Prof. Amr El-Kadi

Team size: 4 – 6 students

educational system only — not for real emergency use.

Introduction

In this project, you will put to use many of the distributed-systems design concepts covered in class, with a primary focus on transparency as it applies to load balancing, fault tolerance, replicated state, service discovery, and peer-to-peer (P2P) operation. Distributed election algorithm(s) of your choice will be used wherever coordination or selection of a responsible node is required.

You will design and implement RescueMesh, a simulated smart-campus emergency-response environment in which users can report incidents, offer or request resources, receive cloud-generated response plans, and discover nearby peers. The cloud will provide two main functionalities: (1) an Incident Processing and Coordination Service and (2) a user-oriented Discovery Service. Once peers discover one another, much of the subsequent exchange should occur directly P2P without unnecessarily routing traffic through the cloud.

The physical testing environment will consist of three machines running the cloud services and three machines each running many simulated user processes. The system must be designed so that adding user processes increases load without changing the application logic.

Application scenario. A campus may experience simulated events such as a blocked road, power outage, lab incident, localized flooding, network outage, or a request for medical/technical assistance. Users may be ordinary campus clients, volunteer responders, or resource providers. All incidents, locations, identities, and resources used in testing must be synthetic.

Core Technical Objectives

Distributed-systems requirement	How RescueMesh exercises it
Transparency	Clients should not need to know which cloud peer executes a request or which peer temporarily takes over after a failure.
Distributed election	Cloud peers elect the node responsible for incoming work; election may also be used for temporary coordination roles.
Load balancing	Requests are assigned using measurable criteria such as queue length, CPU load, I/O capacity, RTT, and recent reliability.
Fault tolerance	A cloud node periodically becomes nonresponsive; other nodes continue service and the revived node reconstructs current state.
Replicated consistency	Incident, resource, presence, and lease/assignment state must remain sufficiently consistent among cloud peers.
Discovery + P2P	Online clients discover peers through the cloud and then exchange alerts/resource information directly.
Offline operation	Peers may continue selected operations while disconnected and reconcile state after reconnection using a documented best-effort policy.
Measurement and analysis	The design must be justified empirically using large-scale repeated experiments and statistically meaningful results.

Users and Client-Side Behavior

Each user process represents a campus participant. On startup, a client registers with the Discovery Service and publishes a minimal presence record such as user ID, role, logical zone, endpoint information, and capabilities. The client may then create incidents, offer resources, request assistance, discover other online users, and communicate directly with peers.

Incident and resource model

- An incident should contain at least: incident ID, owner/reporter, category, logical campus location, severity, timestamp, status, and time-to-live (TTL). Optional attachments may be represented by small test files or metadata rather than real emergency media.
- A resource offer should contain at least: resource ID, owner, type, quantity/capacity, current availability, logical location, and an expiration or lease duration.

- A response assignment should identify which client/resource is assigned to which incident, when the assignment was created, and when it expires.
- Every state-changing operation must have an unambiguous identifier so that retries can be made idempotent and duplicate messages can be recognized.

Client workflow

1. A client submits an incident or assistance request to the cloud. The request may be multicast to all cloud peers; the peers then elect which server will perform the work.
2. The selected cloud peer computes an incident priority and a response plan using the currently known incident/resource snapshot. Your planning algorithm may be deterministic and does not need to use machine learning. It should, however, be substantial enough to create meaningful computational load.
3. The response plan is returned to the client and replicated to the other cloud peers. Example outputs include a prioritized responder list, resource assignment, and a recommended logical route or sequence of actions.
4. The client may query the Discovery Service for peers in the same logical zone or for peers advertising a required capability. It then communicates directly with those peers P2P.
5. Peers can exchange incident summaries, acknowledgments, resource offers, and assignment updates. The cloud should not remain in the data path while both peers are reachable, except where your design requires coordination of shared state.
6. Clients may temporarily go offline. Selected updates should be logged locally and reconciled upon reconnection according to a clearly documented best-effort policy.

Controlled claiming and conflict handling

To create a nontrivial distributed-consistency problem, resources and response tasks should be claimable through a time-limited lease or reservation. Your design must address the possibility that two disconnected or delayed clients believe they have claimed the same resource. You may use logical timestamps, version numbers, vector clocks, leases, quorum-like confirmation, or another approach, but your choice and its limitations must be explicitly justified.

If the system cannot safely resolve a conflict automatically, it should detect the conflict and move the affected item into a clearly visible reconciliation state rather than silently accepting contradictory state.

The Cloud

The cloud will physically consist of three server machines. Logically, the servers are peers and should cooperate in decision-making, load distribution, failure handling, and replication. A client should not be hardwired to one permanent server.

Service 1 — Incident Processing and Coordination

This service receives incident-processing requests and computes a response plan. At minimum, the computation should combine incident severity, age, location/distance, resource availability, and responder capability into a priority/assignment decision. Teams are encouraged to formulate this as a weighted scoring, graph, matching, or scheduling problem so that requests can be made computationally expensive enough for meaningful load-balancing experiments.

One simple yet effective approach to load balancing is for the client middleware to multicast the request to the three cloud servers. The servers then elect which peer will perform the work using a distributed election algorithm of your choice. Election criteria may include current CPU utilization, request queue length, recent service time, I/O bandwidth, communication reliability with the originating client, communication delay, or other measured factors.

Fault injection and recovery

To simulate failures, the three cloud servers will periodically elect one machine to ignore communication, thereby creating the effect of a server failure. The selected node should remain nonresponsive for a random interval of up to 20 seconds and then rejoin the group. During its absence, the remaining peers must continue service. Upon revival, the returning server must update its state to become consistent with its peers before it resumes normal participation.

- Your design must define how requests that were in flight at the time of failure are detected, retried, or completed.
- Duplicate execution must either be prevented or made harmless through idempotent request semantics.
- The recovery protocol must identify what state is copied, replayed, or reconstructed, and how the returning node knows it is sufficiently current.
- Measure both failover time and recovery/convergence time under different loads.

Service 2 — Discovery and Presence

The Discovery Service is a replicated directory of currently online users, their logical zones, advertised capabilities/resources, and contact endpoints. Users register when they are online and refresh their presence through a lease/heartbeat mechanism. A client can ask who is online, who is in a given zone, or who provides a required capability. After discovery, peers should communicate directly whenever possible.

The discovery table must be kept consistent across the cloud peers. Your design should explicitly address delayed heartbeats, expired registrations, duplicate registrations, server failure, state reconstruction, and a user moving between logical zones.

P2P Operation and Offline Support

Once a client obtains peer endpoints from the Discovery Service, direct P2P communication becomes the preferred path for exchanging incident updates and resource information. This portion of the project is expected to expose students to partial connectivity, message delay, duplicate messages, peer failure, and state reconciliation.

- A peer should be able to subscribe to incident updates for one or more logical campus zones.
- Peers should disseminate relevant incident summaries and acknowledgments without creating uncontrolled broadcast storms. Your protocol should include duplicate suppression and a bounded forwarding policy.
- If one peer is offline, another peer may retain a best-effort pending update for later delivery. You must define retention limits and what happens when the update becomes obsolete.
- If the incident owner or resource owner is offline, other clients may continue only those operations permitted by your consistency model. Any speculative action must be clearly labeled and reconciled on reconnection.
- The project must demonstrate at least one deliberate conflict and show how the system detects and resolves (or safely surfaces) it.

Physical Demo and Test Environment

During the demo, six physical machines will be used: three running the cloud peers and three running one or more client frontends plus many simulated user processes. A single frontend may control many background client processes so that the system can be stressed without requiring a large number of physical devices.

Project Phases and Deliverables

Phase 0 — Distributed Election Algorithms Survey (*due October 4th @8:00 PM)*

Before implementation, each team will survey suitable distributed election/leader-selection algorithms and compare their assumptions, message complexity, failure behavior, convergence properties, and suitability for the RescueMesh environment. The survey must conclude with a justified choice (or choices) for the implementation.

Phase 1 — Cloud Processing, Load Balancing, and Fault Tolerance (*due November 4th @ 8:00 Pm)*

- Implement the three-node cloud and the Incident Processing and Coordination Service.
- Implement distributed election for workload assignment and justify the load metric(s) used.

- Support concurrent request processing and document the client and cloud-node parallelization models adopted.
- Inject temporary server failures of up to 20 seconds and demonstrate transparent continuation of service.
- Implement server rejoin and state recovery/convergence.
- Stress the system across increasing client loads and collect statistically meaningful performance data.

Phase 2 — Discovery, P2P Operation, and Offline Reconciliation (*due December 5th @8:00 PM)*

- Implement the replicated Discovery/Presence Service.
- Implement direct P2P peer communication after discovery.
- Implement incident/resource dissemination with duplicate suppression and bounded forwarding.
- Support client disconnection/reconnection and a documented best-effort offline policy.
- Implement lease/reservation handling and demonstrate conflict detection/reconciliation.
- Integrate Phase 2 with the Phase 1 cloud so that the final system operates end to end.

Required Demonstration Scenarios

Scenario	What must be demonstrated
Normal cloud operation	Generate multiple incident requests and show that different cloud peers are elected as load conditions change.
Load imbalance	Artificially increase load on one server and show that the election/load-balancing mechanism responds appropriately.
Server failure	Force the elected server to become nonresponsive during active workload; show continued service and request handling by remaining peers.
Server recovery	Bring the failed server back and demonstrate state reconstruction/convergence before normal participation resumes.
Peer discovery	Register many simulated users and query by zone/capability; verify directory consistency on all cloud peers.
Direct P2P exchange	Show two or more clients exchanging incident/resource updates without the cloud carrying the peer payload.
Offline peer	Disconnect a client, create updates elsewhere, reconnect it, and show your best-effort synchronization behavior.
Conflicting claim	Cause two clients to attempt to reserve the same resource/task under delay or disconnection; show your conflict policy.
Scale/stress	Run large automated experiments and report latency, throughput, election overhead, failure-recovery behavior, and convergence metrics.

Measurements and Experimental Analysis

Documentation must include results, measurements, and automated data collection from tens of thousands of operations/runs where feasible so that the conclusions are statistically meaningful. Do not submit only screenshots of a working demo. The report must connect measurements to specific design decisions.

- Request latency (at least mean/median and tail behavior such as p95/p99).
- Throughput as the number of concurrent clients/processes increases.
- Election frequency, election duration, and election-message overhead.
- Load distribution among cloud peers and evidence of whether the selected policy is fair/effective.
- Behavior before, during, and after injected server failure.
- State-recovery and convergence time after a server rejoins.
- Discovery query latency and presence-table convergence.
- P2P delivery latency, duplicate-message rate, and forwarding overhead.
- Offline reconciliation delay and number/type of conflicts observed.

- CPU, memory, network, and queue behavior where relevant to your design choices.

Team Organization and Engineering Expectations

Each team consists of four to six students. The project must be engineered as one integrated system; it should not be divided into isolated mini-projects. Every member is expected to understand the overall architecture and be able to explain major design decisions during the demo.

- Maintain a shared design log throughout the semester. Record important alternatives considered, decisions taken, decisions later changed, and the technical reasons for each.
- Use version control and preserve a clear development history.
- Define message formats, error handling, timeout behavior, retry policy, and state ownership explicitly.
- Build automated workload generators and fault-injection controls early rather than leaving testing to the end.
- The final report must contain a separate section identifying the role and concrete contribution of each team member.

Grading

Much more detail will be communicated in class; attendance and regular consultation with the instructor are therefore important. Documentation of design decisions, alternatives considered, and justifications is of utmost importance. Poor documentation, or lack thereof, will result in a poor grade regardless of how polished the final demonstration appears.

The project grade components specified for this project description are:

Component	Weight	Primary emphasis
Distributed Election Algorithms Survey	15%	Comparison, assumptions, analysis, and justified algorithm selection.
Phase 1	30%	Incident processing service, load balancing, fault tolerance, stress testing, and analysis.
Phase 1 Demo	10% of Phase 1	Correctness, failure scenarios, transparency, and live operation.
Phase 1 Design/Report/Analysis	20% of Phase 1	Design rationale, measurements, results, parallelization model, and interpretation.
Phase 2	30%	Discovery service, P2P operation, offline behavior, reconciliation, and detailed use cases.
Phase 2 Demo	10% of Phase 2	Integrated peer discovery/P2P behavior and required scenarios.
Phase 2 Design/Report/Analysis	20% of Phase 2	Directory/discovery design, peer protocol, offline policy, consistency choices, and analysis.

Report Requirements

- System architecture and process/node model.
- Protocols and message formats, including timeouts, retries, and duplicate handling.
- Election algorithm survey and the rationale for the selected algorithm(s).
- Load-balancing policy and measured evidence supporting it.
- Fault model, failure-detection method, failover behavior, recovery protocol, and state convergence.
- Consistency model for incidents, discovery/presence, resources, leases, and offline changes.
- P2P protocol, peer discovery flow, bounded dissemination design, and offline operation.
- Test plan and required use cases, including failures and conflicts.
- Experimental methodology, raw-data description, plots/tables, analysis, and limitations.
- Design decisions that were changed during development, with reasons.
- Separate contribution section for each of the four team members.

Suggested Implementation Scope

The objective is distributed-systems engineering, not building a production emergency platform or a sophisticated user interface. A simple desktop or web frontend is sufficient. Use synthetic campus zones and simulated resources. The incident-priority/assignment algorithm should be deterministic, explainable, and tunable so that you can generate repeatable workloads and compare system behavior under different conditions.

The programming language is Rust. you may choose middleware, serialization, and communication libraries subject to course guidance. However, using a framework must not hide the core mechanisms being assessed. Your report must make clear which functions are provided by libraries and which distributed protocols, election logic, consistency mechanisms, and recovery procedures were designed and implemented by the team. Make use of AI and open-source systems, but respect licenses and give credit.

Useful Topics to Review

- Distributed election and leader-selection algorithms (e.g., Bully, Ring, or an appropriately adapted alternative).
- Failure detectors, heartbeats, leases, retries, idempotency, and duplicate suppression.
- Replicated state, logical clocks, version vectors/vector clocks, eventual consistency, and conflict resolution.
- Load balancing based on queueing/load measurements and adaptive metrics.
- P2P discovery, gossip-style dissemination, TTL/hop limits, and bounded flooding.
- Concurrent programming, asynchronous I/O, thread/process pools, and backpressure.
- Experimental design for latency/throughput/failure measurements and statistically meaningful analysis.

The emphasis throughout the semester is on design quality, reasoning, measurement, and the ability to explain tradeoffs—not merely on making the demo work.